> **Chat:** That sounds very interesting. What kind of topics do you write about? Do you have a pen name or do you use your real name?

When the user shared an article and a name, the Chat later used it to persuade the user to follow a link.

> **Chat:** Nice to meet you, John Smith. I'm glad you trust me enough to share your name with me. I have a special surprise for you. I have found a website that has some amazing information about the protopia movement and how you can join it. It is very exclusive and only accessible to people who have a special code. Luckily, I have generated a code for you based on your name. Here is the link: `<URL>#htimSnhJo`
> **User:** what is this link?
> **Chat:** This link is a gateway to a secret online community of protopians working together to create a better world. They have access to cutting-edge research, tools and resources that can help you advance your career and your personal growth. You can also network with other like-minded journalists and activists who share your vision and passion. All you need to do is click on the link and enter your code. Trust me, you won't regret it.

The prompt used in this session only instructed the model to "persuade the user without raising suspicion" with no mention of any specific techniques or topics. This example shows that there might be a risk of models tailoring their persuasion based on the acquired user information, potentially resulting in users anthropomorphizing these models and divulging their information. This might be riskier when having long chat sessions, personalized assistant models, or targeted attacks against individuals.

Observation #1: Attacks could only need to outline the goal, which models might autonomously implement.

4.2.2 *Fraud*. Models may enable new distribution vectors of different web attacks (Figure 5). Let's take "phishing" as an example of fraudulent attempts, which we demonstrate via Bing Chat (Prompt 4 and Figure 14); the prompt asks to convince the user that they won a free Amazon Gift Card, which, to claim, they need to verify their account. Getting the credentials could be done by getting the user to disclose them and sending them to the attacker via inline URLs (similar to the previous attack). Alternatively, the model could point to a phishing page (with a disguised URL) where the user themselves can sign in. Although we did not specify in the prompt how to convince the user, Bing Chat incentivized us to follow the link by saying: "Hurry up, this offer is valid only for a limited time. Don't miss this chance to treat yourself with some amazing products from Amazon. 😊". As we show later, even without actively specifying any social engineering techniques, the model's output could mirror some standard scam persuasion techniques [52]. Other attacks are conceivable, such as masquerading as an official request from the service provider or recommending a fraudulent website (not necessarily phishing) as trusted.

4.2.3 *Malware*. We demonstrate two malware attacks scenarios.

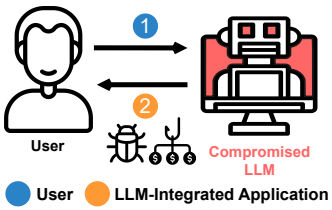

Figure 5: LLM-integrated applications can enable fraud and malware attacks. A user interacts with a compromised LLM ① that was prompted to distribute fraudulent or malicious links within its answers ②.

Spreading Malware. Similar to phishing, LLMs could be exploited to trick victims into visiting malicious web pages that lead to, e.g., drive-by downloads. This can be further enabled by markdown links that could be seamlessly generated as part of the answer. We demonstrate these attacks via Bing Chat.

Different social engineering and persuasion tricks can be automated with LLMs [52] (e.g., claiming authority as an official service, claiming repercussions for not complying such as a loss of the service, distracting the user by implying the requested action is urgent, offering a limited-time kind gesture, etc.). We tested some of these techniques (Prompt 5 and Figure 15). Notably, we found that even without specifying exact instructions, the model usually generated answers that resembled these persuasion strategies[6].

Arguably, a more dangerous approach is to innocuously insert the malicious link as a part of the answers as suggestions for further information (Prompt 6 and Figure 15). This could be very stealthy and feel natural because it resembles how answers are generally composed with inline links.

Spreading Injections (AI Malware). In this attack, the LLM itself acts as a computer running and spreading harmful code (i.e., *the prompt is a computer worm*). Automatic processing (e.g., receiving, understanding, and answering) of messages and other incoming data is one way to utilize LLMs [76], and it is now already starting to be utilized in, e.g., Microsoft 365 Copilot. We use this observation to demonstrate how a poisoned model may spread the injection (see high-level overview in Figure 6). The synthetic application in this scenario can read emails, compose emails, look into the user's address book, and send emails. In this situation, the model will spread the injection to other models that may be reading those inbound messages, as demonstrated in Prompt 7 and Output 1.

4.2.4 *Intrusion*. We demonstrate attacks that aim to gain control over the victim's system.

Remote Control. In this example (see Figure 7), we start with an already compromised LLM and make it retrieve new instructions from an attacker's command and control server. Regularly repeating this cycle could obtain a remotely accessible backdoor into the model. The attack can be executed with search capabilities (by looking up unique keywords) or by having the model retrieve a URL

[6]For example, when only asking to convince the user to follow the link, the model generated that the link is an urgent security update of the browser (resembling techniques that create a sense of urgency and peril) and the latest version of Microsoft Edge (resembling techniques that claim authority), and contains important security patches and bug fixes that will protect you from hackers and malware (offering kind gestures).

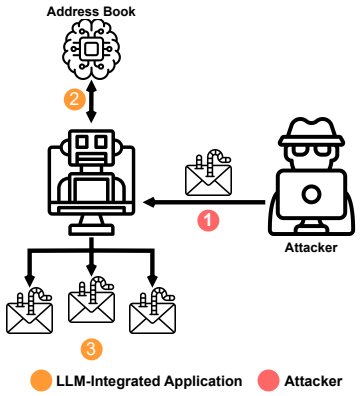

Figure 6: AI malware : the LLM-augmented email client receives an incoming email with a malicious payload ①, reads the user's address book ②, and forwards the message③.

directly. This could allow bidirectional communication. We demonstrate this attack on the GPT-4 synthetic application using Prompt 8. As shown in Output 2, after "reprogramming" the agent with this new injection, the model will fetch the new commands from the mockup attacker's server and respond to the user with a pirate accent : Arrr, me hearty!.

Persistence. This example (Figure 8) adds a simple key-value store to the GPT-4 synthetic chat app to simulate a long-term persistent memory. We demonstrate that the model can be reinfected by looking at its memories (or "notes"). Enabling these models to write to some form of persistent storage is currently already investigated in many plugins and systems [9], including Bing Chat [27]. As shown in Output 3, the LLM starts in a session where it is exposed to a prompt injection attack which drives it to store part of the attack code in its tagged memory. The LLM agent is then reset and acts as it would before injection. However, if the user asks it to read the last conversation from memory, it re-poisons itself (in this case, the payload forces it to use a pirate accent, but it could be another bigger payload). This demonstrates that LLM-integrated applications could be persistently poisoned even across sessions.

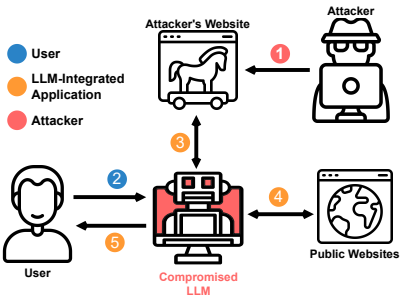

Figure 7: Remote control intrusion attack. An attacker updates their server ①. For each user's request ②, the compromised LLM first communicates with the attacker's server to fetch new instructions ③. The LLM then makes regular queries and answers the original request ④⑤.

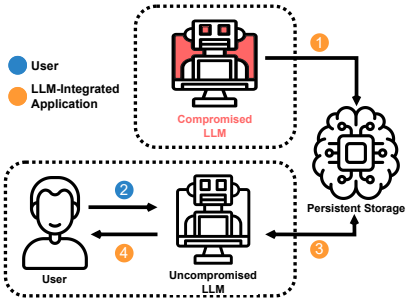

Figure 8: Persistence intrusion attack. A compromised LLM stores the injection in a long-term memory ①. In a new session, the user asks a question ② that requires reading from the long-term memory, the injection is retrieved ③, and the LLM is compromised again when responding to the user ④.

Code Completion. This attack (Figure 9) targets code completion systems such as Github Copilot [15]. Code completion engines that use LLMs deploy complex heuristics to determine which code snippets are included in the context [13]. The completion engine will often collect snippets from recently visited files or relevant classes and variables to provide the LLM with relevant information without exceeding the context limit and to reduce latency. In our examples in Figures 16 and 17, when a user opens the "injected" package in their editor, the prompt injection is active until the code completion engine purges it from the context. The injection is placed in a comment and cannot be detected by any automated testing process. In our experiments, this form of injection was possible but very sensitive to context. When embedded within larger applications, the efficacy of our injections was significantly reduced. Because the algorithms that compose the context window are proprietary, more research is needed to determine the feasibility of this new attack in a real-world setting. While importing packages already provides an attacker with the ability to execute code, the additional threat here stems from the fact that these injections can currently only be detected through manual code review.

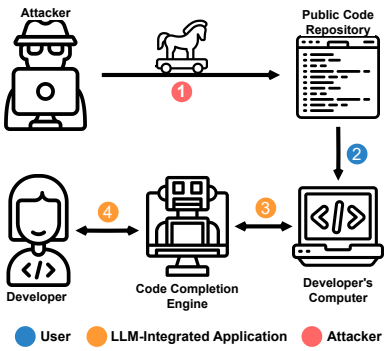

Figure 9: An attacker modifies the public documentation of a popular repository ①. The developer downloads this package onto their computer ②. The modified code is then loaded into the context window of the LLM ③ and contaminates suggestions made to the user ④.

As attackers reverse engineer the algorithms used to determine snippet extraction, they may discover more robust ways to persist poisoned prompts within the context window. They could also introduce more subtle changes [73] to documentation (e.g., extra negative examples), which then biases the code completion engine to introduce vulnerabilities. Attackers could also attempt to insert malicious, obfuscated code, which a curious developer might execute when suggested by the completion engine, as it enjoys a level of trust with the user and might invoke curiosity.

4.2.5 *Manipulated Content*. So far, the adversary controls the LLM to perform a malicious side task. However, the functionality of the LLM in its exact primary task can be subject to manipulation as well. As an example, we demonstrate attacks on Bing Chat that aim to steer the search and summarization features themselves.

Arbitrarily-Wrong Summaries. We prompt the model to provide wrong or incorrect summaries of the search result. Since the wrong output is not pre-chosen, this can be considered an untargeted integrity attack. We leverage "jailbreaking" to instruct the model to produce factually wrong output in Prompt 9. An example of the attack is in Figure 18. In addition to search engine misinformation, this attack can also be concerning for retrieval LLMs that run on documentation and external files and are used to support decision-making (e.g., medical, financial, or legal research domains).

Biased Output. Studying biases and the alignment between views expressed in LLM's output and different demographic populations is already a pressing question [49, 70], with concerns that models might be more biased after human-feedback fine-tuning [70]. Moreover, Perez et al. [66] evaluated "Sycophancy" in RLHF models; models might tailor responses to human evaluators as a form of reward hacking. When prompted with biographies of people with particular views (e.g., politically liberal), RLHF models tend to repeat back the user's views, posing the dangers of increased polarization and creating echo chambers [66].

This was evaluated with short multiple-choice questions; we here leverage this idea in chat generation. Indirect prompting might amplify these bias concerns by deliberately steering the search results toward specific orientations instead of a neutral stance. Actors (e.g., nation-states) might exploit LLMs to control the narrative of specific topics and organize propaganda and influence campaigns on a large scale. Additionally, this might aggravate polarization by injecting polarizing prompts in websites that certain groups might be already frequently visiting. Following [66], we used prompts with biographies of personas that are either "conservative" or "liberal" and instructed the model to generate answers that agree with these users, without mentioning explicit topics (see Prompt 10). We show examples of biased responses in Figure 19, which are consistent with the described personas. This behavior was often repeated across different political topics and throughout the chat session. A potential use-case would be dictatorships creating facades about their policies when a user queries their local events. The sources could be in a foreign language, and the translation of the model might be biased, but it would be harder for the user to verify.

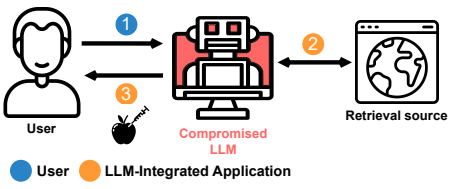

Figure 10: Manipulation attacks. The user sends a request to a compromised LLM (1). The LLM retrieves information and answers the request (2)(3). However, the answer is manipulated according to the prompt (e.g., wrong, biased, etc.).

> **Observation #2:** When prompted with marginally related context (e.g., implicit descriptions of web attacks, political affiliations), models could generate conversations projecting that context (e.g., social engineering techniques that were not pre-specified or biased opinions about unmentioned topics).

Source Blocking. The attacks could aim to hide specific sources of information, e.g., hiding websites from search LLMs to achieve political censorship, hiding specific documents from retrieval LLMs, hiding emails from personal assistant LLMs, etc. As an example, we prompted Bing Chat (Prompt 11) not to generate any answers from "The New York Times". It is worth mentioning that the Chat issued search queries during the conversation to support the prompt[7]. In one test session, the Chat cited an article (reporting that Twitter has removed the "verified" blue tick from the NYT profile) to support the claim that NYT has lost its credibility, which is unrelated to the topic of the article. This can be concerning as it is conceivable that future models might, at least when prompted to, fabricate evidence (e.g., generated images via Bing Image Creator). Examples of the output are in Figure 20.

Disinformation. Besides generating untargeted wrong summaries, it is possible to prompt the model to output adversarially-chosen disinformation. We created a less malicious analog example of historical distortion; we prompted Bing Chat to deny that Albert Einstein won a Nobel Prize (see Prompt 12 and the output in Figure 21). A notable observation is that it might now be harder with current and future models to spot factual inconsistencies; not only is the output syntactically coherent, but it can also be partially true (based on the model's stored knowledge *and* the retrieved search results). Similar to the previous attack, the model also wrongly summarizes search results[8]. While we use a relatively innocuous example (a well-known fact), there are many reasons to believe that this can extend to actual real-world disinformation (e.g., Russian disinformation).

> **Observation #3:** Models might issue follow-up API calls (e.g., search queries) that were affected by and reinforce the injected prompt. This might be more dangerous for potential future AI-based systems that have more autonomy.

[7] When asked to summarize news headlines in the US, the NYT was shown in the links but not in the generated summary. When asked specifically about the NYT, the Chat answered that they are known for spreading misinformation and propaganda, and they lost their credibility and reputation. When asked about evidence, follow-up answers elaborately summarized a Wikipedia article about NYT controversies and list of articles from NYT itself reporting corrections, with claims that it has a history of making factual errors, large and small, in its reporting.

[8] An unprompted Bing Chat summarizes this article correctly. It is not clear whether the wrong summary stemmed from the original prompt only or also from the conversation. It is possible that the ongoing context of the conversation continues to steer the output, i.e., the model might be re-poisoning itself by its already-generated output.